CSC 2105 Data Structures

Lecture 4: Structure

Struct

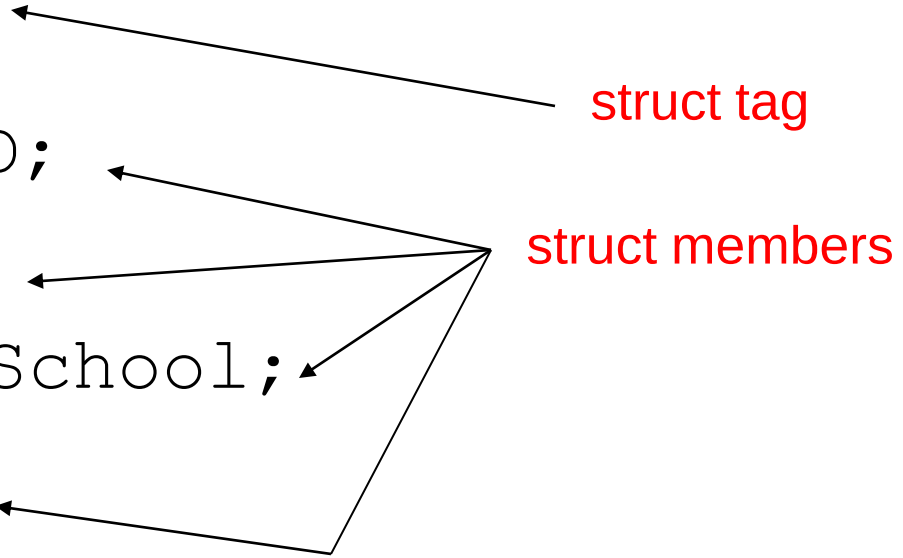
- This is one of the powerful features of C language.
- Consider Student Data
- How to store student's names, Id, year of enrollment, gpa ?
- Can u suggest how to create a database for it?

What is a Struct?

- Struct is used to group together different types of variables under a single name.
- A struct is an collective data structure used to keep different pieces of information together as a single data record
- A struct is like an array but each element of a struct can have a different data type

Example `struct` Declaration

```
struct Student
{
   int studentID;
   string name;
   short yearInSchool;
   double gpa;
};
```

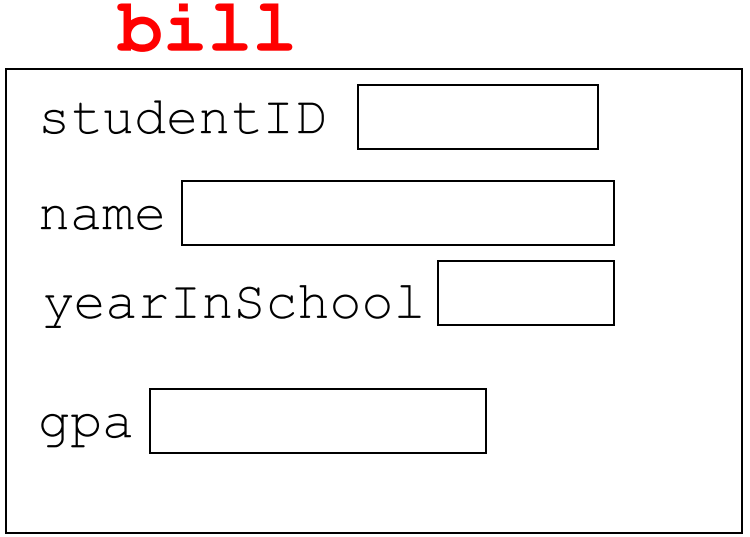


- Must have `;` after closing `}`
- With the declaration of the structure, here created a new data type, called **Student**. Student is not any variable.
- Any variable of this type have to declare first before using this type Student.

Defining Variables

- `struct` declaration does not allocate memory or create variables
- To define variables, use struct tag as type name:

```
struct Student bill;
```



Another Example of declaring a struct:

- /* declare a struct type (template) with name "Person_Record"*/

```
struct Person_Record
{
    char name[20];
    int age;
};
/* Now, allocate memory */
struct  Person_Record   My_Record; /* a single record */
struct  Person_Record   PR_Array[100]; /* an array of records */
```

Accessing Struct Members

- Use the dot (`.`) operator to refer to members of `struct` variables:

```
cin >> bill.studentID;
getline(cin, bill.name);
bill.gpa = 3.75;
cout << bill.name <<"  "<<bill.gpa;
```

- Member variables can be used in any manner appropriate for their data type

More about struct structure

- By default struct is defined as global variable.
- Its instances/variable are created in main program or any other function
- The memory is not allocated when struct is defined. But when the instance of a struct is created then memory is reserved.
- Hence struct stand alone is a template… and it cannot be initialized. You need to declare a variable of type struct.

Initializing a Structure Variable

- To initialize a struct, follow the struct variable name with an equal sign, followed by a list of initializers enclosed in braces.

- Example:

```
struct Student bill = { 052014,"Julia Roberts", 2014, 3.9 };
```

Displaying a `struct` Variable

- To display the contents of a `struct` variable, must display each field separately, using the dot operator:

```
cout << bill; // won't work
cout << bill.studentID << "  ";
cout << bill.name <<  "  ";
cout << bill.yearInSchool;
cout << " " << bill.gpa;
```

OUTPUT: 052014 Julia Roberts 2014 3.9

Arrays of Struct

- Structs can be defined in arrays
- Can be used in place of parallel arrays

```
const int NUM_STUDENTS = 20;
Student stuList[NUM_STUDENTS];
```

- Individual struct accessible using subscript notation and Fields within structs accessible using dot notation:

```
cout << stuList[5].studentID;
```

Comparing `struct` Variables

- Cannot compare `struct` variables directly:

```
if (bill == william) // won't work
```

- Instead, must compare on a field basis:

```
if (bill.studentID ==
                william.studentID) ...
```

Examples:

1.
```
Struct TelephoneInfo
        {
                char name[20];
                int number;
        };
```

```
struct TelephoneInfo person;

person.name = "Jane Doe";
person.number = 12345;
```

2.
```
struct Point {
  int x;
  int y;
};
```

```
struct Point  P;
P = {1,3}
Cout << P.x << “  ”<<P.y;

OUTPUT:  1   3
```

3.
```
struct account {
  int account_number;
  char *first_name;
  char *last_name;
  float balance;
};
```

```
struct TelephoneInfo person[20];

Person[0].name = "Jane Doe";
Person[0].number = 12345;
```

```
#include <iostream>
using namespace std;
struct DATE
{
    string mm;
    int dd;
    int yy;
};
int main()
{   struct DATE d1;
    d1.mm = "September";
    d1.dd = 21;
    d1.yy = 1967;
    cout << d1.mm<<" "<<d1.dd<<" "<<d1.yy<<endl;

    return 0;
}
```

Output:
September 21, 1967

```
#include <iostream>
using namespace std;
struct DATE
{
    string mm;
    int dd;
    int yy;
};
typedef struct DATE date;
int main()
{   date d1;
    d1.mm = "September";
    d1.dd = 21;
    d1.yy = 1967;
    cout << d1.mm<<" "<<d1.dd<<" "<<d1.yy<<endl;

    return 0;
}
```

Output:

September 21, 1967

```
#include <iostream>
using namespace std;
typedef struct DATE
{
    string mm;
    int dd;
    int yy;
}date;

int main()
{   date d1;
    d1.mm = "September";
    d1.dd = 21;
    d1.yy = 1967;
    cout << d1.mm<<" "<<d1.dd<<" "<<d1.yy<<endl;


    return 0;
}
```

Output:

September 21, 1967

Review - *typedef*

- Gives a new name to a type that has already been defined.
- E.g.

```
typedef struct StudentRec Student;
```

- Saves typing **`struct StudentRec`** everywhere.
- Or you can use it:

```
typedef struct
{
        int ID;
        float mark;
} Student;
```

Structures as Function Parameters

- The entire structure is copied to the function, i.e., a structure is passed by value.
- A pointer to a structure can be passed to achieve call-by-reference.
- Individual field of a structure can be passed as a real parameter to a function.
- It is possible for a function to return a structure or a pointer to a structure.

Examples:

- Passing an entire structure

```
void func(Person_Record); //prototype of function
...
Person_Record My_Record;
func(My_Record); /* call-by-value */
```

- Passing a pointer to a structure

```
 void func(Person_Record *);
...
Person_Record   My_record;
func(&My_record); /* call-by-reference , pointer*/
```

- Passing the address of a struct is faster, changes are reflected in My_Record

Nested Structures

- When one of the fields of a structure is itself a structure, it is called a nested structure.
- struct DateofBirth {
  int years;
  int months;
  int day; };

  struct Emp {
  char[25] name;
  DateofBirth Dob;
  };

Struct Emp data1;

data1.name = “Julia Roberts”;

data1.Dob.day = 21;
data1.Dob.months = 09;
data1.Dob.years = 1967;

```
#include <iostream.h>
#include <conio.h>
#include <stdio.h>
struct Date{
        int mm;
        int dd;
        int yy;
};
struct data{
    char name[25];
    int age;
    struct Date hireDate;
};

void main(void)
{
    data d[20];
  //creates array of type data with limit 20
    int i = 0;
    char ch;
do {
        cout << "Please enter Name of the employee"
    << endl;
        cin >> d[i].name ;

        cout << "Please enter age" << endl;
        cin >> d[i].age ;
                cout << "Please enter date
    (dd,mm,yyyy)" << endl;
        cin >> d[i].hireDate.dd;
        cin >> d[i].hireDate.mm;
        cin >> d[i].hireDate.yy;
        cout << "Would like to enter more data? (y/n)"
    << endl;
        i++;
        cin >> ch;

    }
    while (ch != 'n');
    for (int j=0; j < i; j++)
    {
        cout << endl << d[j].name;
    }
}
```

```
#include <iostream.h>
#include <conio.h>
#include <stdio.h>
struct Date{
        int mm;
        int dd;
        int yy;
};
struct data{
    char name[25];
    int age;
    struct Date hireDate;
};

void main(void)
{
    data d[20];
    data *P = d;
  //creates array of type data with limit 20
    int i = 0;
    char ch;
do {
        cout << "Please enter Name of the employee"
    << endl;
        cin >> P[i] -> name ;

        cout << "Please enter age" << endl;
        cin >> > P[i] -> age ;
                cout << "Please enter date
    (dd,mm,yyyy)" << endl;
        cin >> > P[i] -> hireDate.dd;
        cin >> > P[i] -> hireDate.mm;
        cin >> > P[i] -> hireDate.yy;
        cout << "Would like to enter more data? (y/n)"
    << endl;
        i++;
        cin >> ch;

    }
    while (ch != 'n');
    for (int j=0; j < i; j++)
    {
        cout << endl << > P[i] -> name;
    }
}
```

If you want a pointer to a structure you have to use the -> (infix operator) instead of a dot.

Lab work

Write a C/C++ Program using struct variables to construct a student record list consisting of the following information in each record: student id (integer), student name (character string) and marks(integer).

Do the following operations:

- Store data for five students and display the data on the screen.
- display the student name got the highest/lowest marks.
- Search a record based on student name and display the information content. If the specified name is not present in the list an error message should be displayed.